Noisy Neighbour

one tenant's spike must not become everyone's outage

Watch the 4-min video

In one line: stop one tenant from degrading the rest with layers: per-tenant **rate limits** at the edge, **quotas** on expensive resources, **fair scheduling** of background work, **isolation** of heavy tenants in their own pools or shards, and per-tenant monitoring.

1 The Monday import

9 a.m. Monday: one customer starts a bulk import of two million records. Within minutes every other customer's dashboard crawls, API calls time out and support tickets pile up. The importer did nothing wrong; the platform simply let one tenant take far more than its share of CPU, database connections, I/O and queue time. In a pooled system (lesson 41) fairness is not automatic. You have to build it at every shared resource.

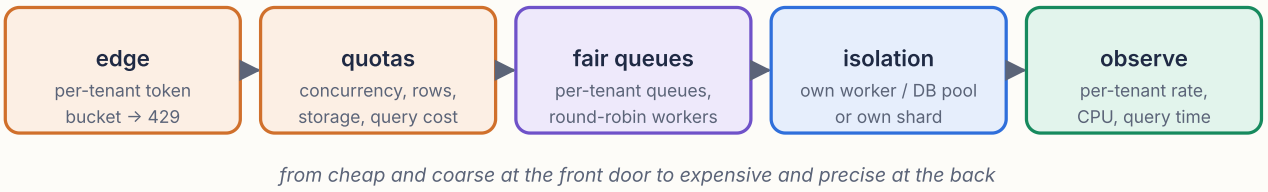


Figure 1. Defence in depth against noisy neighbours.

2 Per-tenant rate limits at the edge

Give each tenant its own **token bucket**: capacity 400 tokens, refilled at 200 per second. A burst of 400 requests passes, then the tenant is held to 200 per second on average. A request that finds the bucket empty gets **429 Too Many Requests** (RFC 6585) with a `Retry-After` header, so well-behaved clients back off. Limits come from the tenant's plan (lesson 49): enterprise gets more than free, nobody gets unlimited.

- **Key by tenant and by user or API key**, so one script inside a tenant can't use the whole tenant's budget.
- **Weight by cost.** A search or export costs more tokens than a simple read; counting only requests lets one expensive endpoint slip through.
- **Distributed counters.** Many gateway nodes share a limit through Redis (an atomic Lua script per check), or each node enforces limit ÷ nodes locally, trading accuracy for zero extra latency.

3 Quotas on expensive resources

Rate limits protect the front door; the heavy work happens behind it. Add quotas on what is actually scarce: **concurrent jobs** per tenant, rows per import, storage, exports per hour, and query cost. In the database, a per-role or per-session `statement_timeout` stops one runaway report from holding locks and I/O for an hour. **Soft limits** warn (email, banner, header); **hard limits** stop or queue the work. Quotas turn a sudden spike into a bounded, predictable load, and the same usage numbers feed billing (lesson 45).

4 Fair scheduling for background work

A single shared FIFO queue is unfair by design: a tenant that enqueues 10,000 jobs puts everyone else behind them. Keep a **queue per tenant** (or a partition per tenant group) and let workers pick across tenants **round-robin**, or **weighted fair** so paid tiers get a bigger share. Fairness doesn't waste capacity: when other tenants are idle, the big tenant still gets every worker.

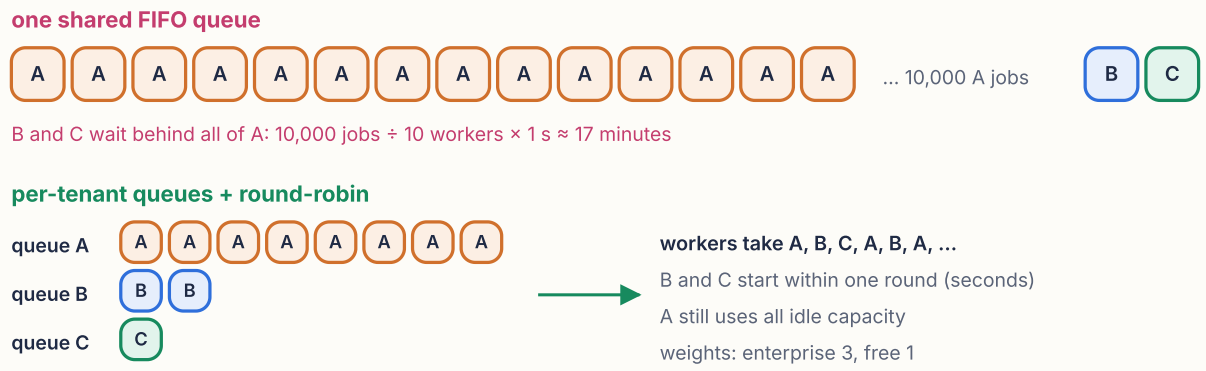


Figure 2. Same jobs, same workers. Only the pick order changed, and B's wait drops from minutes to seconds.

Implementation sketch: a Redis list per tenant plus a set of tenants with pending work; each worker rotates through that set, pops one batch, and re-adds the tenant if work remains. With Kafka, keying by tenant gives per-tenant ordering but a hot tenant still overloads its partition, so large tenants are usually split out to their own topic or consumer group.

5 Isolate heavy tenants

Some tenants are simply big. Use the **bulkhead** pattern: give the heaviest tenants their own worker pools, their own database connection pool (so they can exhaust only their own connections), or their own shard or database (lesson 42). **Shuffle sharding**, described in the AWS Builders' Library, is a clever middle ground: each tenant is assigned a small random subset of workers, so a noisy tenant can take down only its own subset, and almost no other tenant shares exactly the same subset.



Figure 3. Shuffle sharding. With plain sharding into 4 pairs, X's whole pair (a quarter of tenants) would suffer; with shuffle sharding, Y survives.

Tenant **placement** then becomes an operational lever: spread tenants across cells or shards so one noisy tenant affects only its neighbours, move a tenant that grows too large to a dedicated shard, and at the top end, give it a dedicated stamp (lesson 50).

6 Watch per-tenant metrics

You can't fix a neighbour you can't see. Track request rate, error rate, CPU time, database time and queue depth **per tenant**, and alert on any tenant whose share jumps. Careful with cardinality: a metrics label per tenant across 50,000 tenants overwhelms Prometheus-style systems, so label the top tenants

(say top 100 by usage) individually and aggregate the rest, and keep full detail in logs or the metering pipeline.

WORKED EXAMPLE: THE BULK IMPORT, REDESIGNED

The import API now accepts the file, stores it in object storage and returns `202 Accepted` with a job ID. The file is split into batches of 5,000 rows (400 batches for 2 million rows) placed on the tenant's own queue. Workers pick batches by **weighted round-robin** across tenants, the importer is capped at **4 concurrent batches**, and its writes go through a separate, small database connection pool. Edge limits still return 429 with Retry-After if its scripts poll too hard. The import takes a little longer; nobody else notices it.

TRADE-OFFS

- Strict limits protect neighbours but frustrate big customers; tie them to plans and offer higher tiers.
- Fair queues add bookkeeping (many queues, rotation) compared with one FIFO.
- Dedicated pools isolate well but waste idle capacity; shuffle sharding isolates with shared capacity.
- Local, per-node limits are fast but approximate; central counters are exact but add a network hop.

COMMON MISTAKES

- Global rate limits only: one tenant uses the whole global budget and everyone gets 429s.
- Counting requests, not cost, so a cheap-looking export endpoint melts the database.
- One shared job queue in arrival order.
- Shared connection pools, so one tenant's slow queries starve everyone of connections.

7 Key terms

Noisy neighbour: one tenant degrading others on shared resources

Quota: cap on a resource: soft warns, hard blocks

Bulkhead: separate resource pools so failures stay contained

Token bucket: rate limit with a refill rate and burst capacity

Fair scheduling: giving every tenant a turn, optionally weighted

Shuffle sharding: each tenant on a random small subset of workers

8 Interview questions

Q1. How do you stop one tenant from degrading everyone? BASIC

Per-tenant rate limits at the edge, quotas on expensive resources, fair scheduling with per-tenant queues, isolation of heavy tenants in their own pools or shards, and per-tenant monitoring.

Q2. Why isn't a single shared queue enough? BASIC

One tenant's backlog blocks everyone behind it; per-tenant queues with round-robin or weighted fair picking give every tenant a turn.

Q3. What does a 200/s rate with a burst of 400 mean? BASIC

A token bucket of capacity 400 refilled at 200 tokens per second: a 400-request burst passes, then the tenant averages at most 200 requests per second.

Q4. One tenant's reporting queries slow the shared database at night. What do you do? SCENARIO
Add per-tenant query timeouts and concurrency limits, move its reports to a read replica or warehouse, give it its own connection pool, and if it keeps growing, move it to its own database.

Q5. How would you find which tenant is causing a slowdown? SCENARIO
Per-tenant metrics on request rate, CPU and database time, with the top tenants labelled individually, plus tenant IDs in traces and slow-query logs.

Q6. Why limit by cost rather than request count? TRICKY
Requests differ by orders of magnitude in cost; counting them equally lets a tenant stay under its request limit while saturating the database with expensive calls.

Q7. What's the advantage of shuffle sharding over plain sharding? TRICKY
With plain sharding a noisy tenant hurts everyone in its shard; with shuffle sharding few or no tenants share its exact worker set, so most still have a healthy worker.

Next: SD-R-005 · Usage Metering backendroadmap.com